Data Availability Statement

For the manuscript: "A nuclear-register criterion for radical-pair computation in cryptochrome"

Hikaru Wakaura and Taiki Tanimae, QIRI (Quantum Integrated Research Institute Inc.), Tokyo, Japan

h.wakaura@qiri.co.jp, t.tanimae@qiri.co.jp

Code. All results in this work are produced by the open-source Python package `qbscreen` (MIT licence), available at https://github.com/deeptell-inc/new_Q_brain. The package implements the exact Liouville-space Haberkorn master equation with electronic decoherence and the correlated-pair-birth quantum-reservoir-computing protocol. The specific modules are:

- `qbscreen/master_equation.py` — Liouville-space solver and its validation against the closed-form coherent limit (supplementary material, S1);
- `qbscreen/reservoir.py` — reservoir Hamiltonian and propagator, and the out-of-sample information-processing capacity (IPC) and memory capacity (MC) estimators (trained on the first half of each run, scored on the held-out second half);
- `qbscreen/corrected_injection.py` — the chemically consistent correlated-pair birth channel $\rho_{\mathrm{e}}(s) = s\,|S\rangle\langle S| + (1-s)P_{\mathrm{T}}/3$ and its S–T_0 coherent variant, together with the single-electron-reset control that reproduces the artifact it replaces;
- `qbscreen/readout_routes.py` — the seven biologically accessible readout routes with recombination active, the product-weighted CIDNP observable, the turnover clock, and the nuclear-register depolarisation parameter q;
- `qbscreen/final_numbers.py` — protocol-consistent recomputation of the engineered-point capacity, coherence fraction, classical baselines and T_2^e scan; the chemical-readout, clock and criterion headline numbers come from `readout_routes.py` and `panel_response.py`;
- `qbscreen/panel_response.py` — the controls: the product-state register of Eq. (2), the per-route memoryless floors at $q = 1$, the delay-resolved memory kernel, the twelve-seed turnover-clock scan with paired statistics, the grid convergence of MC, the depolarising-versus-dephasing nuclear channel comparison, the cryptochrome-point classical baselines, and the ridge and shot-noise sensitivity scans. Outputs are written to `simulation_results/panel/`;
- `qbscreen/qrc_benchmarks.py` — classical echo-state-network baselines, the NARMA2 benchmark, and the coherence-time scan;
- `qbscreen/semiclassical.py` — the classical-spin reference model for the coherence fraction (supplementary material, S8);
- `qbscreen/nuclide_register.py` — per-nucleus register relaxation, i.e. whether a proton-only register suffices (supplementary material, S9);
- `qbscreen/ensemble_pooled.py` — pooled ensembles with heterogeneous couplings, desynchronised turnover, and heterogeneous relaxation times with the mean-field control that separates the spread from the shift in mean survival (supplementary material, S10);
- `qbscreen/general_spin.py` — anisotropic hyperfine tensors and ^{14}N as spin-1 (supplementary material, S11);
- `qbscreen/relaxation_estimate.py` and `qbscreen/premise_dependence.py` — every cryptochrome-point quantity rerun with the electron–electron couplings of Efimova and Hore (supplementary material, Sec. S14); `qbscreen/turnover_estimate.py` — prediction of the two constants the criterion depends on, with the relaxation calculator validated against three measured systems (supplementary material, S12);

- qbscreen/product_carrier_audit.py — the sensitivity analyses (nuclide restriction, desynchronisation, anisotropy) with the product register (supplementary material, S4);
- qbscreen/proton_register_reuse.py — the register the biology leaves: ^{14}N wiped every cycle, product carried over, proton partially relaxed; the ceiling and boundary recomputed on that curve (open8_proton_register_reuse.json).
- qbscreen/counting_noise.py — the eight CIDNP-route channels sampled as a pool of N molecules is counted, against the relative-Gaussian model (open9_counting_noise.json).
- qbscreen/semiclassical_trajectories.py — trajectory-number convergence of the semiclassical reference and its floor (open10_semiclassical_trajectories.json).
- qbscreen/ridge_cv_comparison.py — the channel-matched quantum-versus-ESN comparison with the ridge chosen by cross-validation (open11_ridge_cv_comparison.json).
- qbscreen/closed_cycle.py — the product and unreacted branches summed by molecule number before the register is handed on (open12_closed_cycle.json).
- qbscreen/input_carrier.py — the input carried by the field magnitude, the singlet recombination rate or a small-signal singlet fraction (open13_input_carrier.json).
- qbscreen/axial_tensor_model.py — the flavin nitrogens as axial tensors built from the characteristics stated by Lee *et al.* (2014), with the field orientation scanned and averaged (open14_axial_tensors.json).
- qbscreen/published_tensors.py — the published $FAD^{\bullet-}$ and $TrpH^{\bullet+}$ hyperfine tensors (Hiscock *et al.* 2016, SI Appendix Tables S1 and S2) on the three nuclei of the main model with the field orientation scanned, and with a fourth nucleus (open17_published5.json, open17_published6a.json, open17_published6b.json).
- manuscript/make_schematic.py and manuscript/make_fig_criterion.py — Figs. 1–3.

Data. All numerical outputs underlying the figures and tables are provided as JSON files in simulation_results/ of the repository, principally final_numbers.json, corrected_injection.json, readout_routes.json, register_reuse.json, grid_convergence.json, qrc_benchmarks.json and cryptochrome_reality.json, together with the control and sensitivity outputs in simulation_results/panel/ (c1–c6, m6–m9, open1–open17, s2 and audit3). Three drivers—ensemble.py, quantum_vs_classical.py and run_reservoir in reservoir.py—implement the single-electron reset of supplementary material S3 and are retained as controls only; no reported result derives from them, though the capacity estimators in reservoir.py are used throughout. No experimental data were generated; the study is entirely computational. Every reported capacity is out of sample, is quoted with its route-specific memoryless floor, and is shown to be converged in sample length and in the time grid; capacities are means over 3–12 input realisations with the stated standard deviations (the ridge and shot-noise sensitivity scans, the semiclassical integration-step check and the single-realisation clock scan of supplementary material Table S11 are single-realisation and are excluded): twelve for the turnover-clock scan, four for the register-reuse and nuclear-channel scans and the semiclassical reference, three for the ensemble and anisotropy scans, and six to eight elsewhere. The turnover-clock scan, which carries the central claim, uses twelve realisations; because the same seeds are used at every pause length, the change in capacity across pause lengths is reported as a paired difference with its standard error.

Reproducibility. The central claims—the agreement of the Liouville-space solver with the coherent limit, the Dambre capacity bound, the correlated-pair-birth readout capacities, and the nuclear-register reuse criterion—are covered by an automated test suite (qbscreen/tests/; 315 tests, all passing: test_panel_claims.py binds the headline quantities and load-bearing claims, and test_-

`table_rows.py` binds the capacity and standard-deviation cells of every table, with the expected values transcribed from the manuscript rather than from the data files, while `test_printed_cells.py` parses the printed cells back out of the `.tex` and checks them against the data, so a drift on either side fails. `test_spin_parameters.py` binds the input-parameter table and `test_cross_document_refs.py` the hand-typed cross-document table references. The tests that read the manuscript run on the LaTeX sources, which accompany this submission together with the freeze manifest rather than being distributed in the public repository; on a public clone those tests skip with that reason, and the collected count is unchanged. The table tests see only tables, so `test_prose_numbers.py` binds the criterion numbers printed in sentences—in this file's companions `README.md`, the claims ledger and the cover letter as well as the manuscript—and fails when a new one appears with neither a binding nor a recorded waiver; derived-time and raw-IPC columns are not yet bound). Every number reported in the paper can be regenerated from a clean checkout with the commands below; `scripts/regenerate_all.sh` runs exactly this list and checks the result byte-for-byte against the shipped files; the three figures are rebuilt with `manuscript/make_schematic.py` and `manuscript/make_fig_criterion.py`. One shipped file, `open5_feasible_region.json`, duplicates a block inside `open5_turnover_estimate.json` and has no separate command.

```
pip install -e .
python -m pytest qbscreen/tests/ -q
python -m qbscreen.final_numbers
python -m qbscreen.readout_routes
python -m qbscreen.readout_routes refined
python -m qbscreen.corrected_injection
python -m qbscreen.reanalysis
python -m qbscreen.panel_response
python -m qbscreen.semiclassical
python -m qbscreen.semiclassical floor
python -m qbscreen.semiclassical conv
python -m qbscreen.nuclide_register
python -m qbscreen.ensemble_pooled
python -m qbscreen.premise_dependence
python -m qbscreen.general_spin
python -m qbscreen.relaxation_estimate
python -m qbscreen.turnover_estimate
python -m qbscreen.product_carrier_audit
python -m qbscreen.proton_register_reuse
python -m qbscreen.counting_noise
python -m qbscreen.semiclassical_trajectories
python -m qbscreen.ridge_cv_comparison
python -m qbscreen.closed_cycle
python -m qbscreen.input_carrier
python -m qbscreen.axial_tensor_model
python -m qbscreen.axial_tensor_model ratio
python -m qbscreen.proton_register_reuse closed
python -m qbscreen.published_tensors published5
python -m qbscreen.published_tensors published6a
python -m qbscreen.published_tensors published6b
python -m qbscreen.reservoir
python -m qbscreen.reservoir realism
python -m qbscreen.qrc_benchmarks
python -m qbscreen.qrc_benchmarks tradeoff
python -m qbscreen.qrc_benchmarks cryptochrome
python -m qbscreen.ensemble
python -m qbscreen.quantum_vs_classical
```

Software. Python 3, NumPy ($\geq$ 1.24), SciPy ($\geq$ 1.10) and Matplotlib ($\geq$ 3.7). No proprietary software

is required.

This statement follows the Royal Society of Chemistry Data Sharing Policy: https://www.rsc.org/journals-books-databases/about-journals/data-sharing-policy/